```c
#include <stdio.h>

struct node {
    int data;
    struct node *next;
};

struct node * find_last_node(struct node *head) {
    if (head == NULL)
        return NULL;
    struct node *p = head;
    while (p->next != NULL)
        p = p->next;
    return p;
}


struct node * find_second_last_node(struct node *head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) return NULL;    // only one node
    struct node *p = head;
    while (p->next->next != NULL)
        p = p->next;
    return p;
}




struct node * find_second_last_node(struct node *head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) return NULL;
    struct node *p = head;
    struct node *q = head;
    while (p->next != NULL) {
            q = p;
            p = p->next;
    }
    return q;
}


struct node * find_kth_last_node(struct node *head) {
    if (head == NULL) return NULL;
    struct node *p = head;
    struct node *q = head;
    int k = 3;

    // move p k nodes ahead
    for (int i = 0; i < k; i++) {
        if (p == NULL) return NULL;
            p = p->next;
    }

```

```c
    // move p and q together
    while (p->next != NULL) {
        p = p->next;
        q = q->next;
    }

    return q;
}



struct node * find_kth_last_node(struct node *head, int k) {
    if (head == NULL) return NULL;
    struct node *p = head;

    while (k != 1 && p->next != NULL) {
        p = p->next;
        k -= 1;
    }

    // if while loop ends and k is not 1, size of list < k
    if (p->next == NULL && k != 1) return NULL;

    struct node *q = head;
    while (p->next != NULL) {
        p = p->next;
        q = q->next;
    }

    return q;
}




// to make the last node the first node

struct node * make_last_node_first(struct node *head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) return head;

    struct node *p = head, *q = head;

    while (p->next != NULL) {
        q = p;
        p = p->next;
    }
    // p becomes last, q becomes second last

    p->next = head;
    q->next = NULL;
    head = p;
    return head;
}
```

```c


struct node * reverse_linked_list(struct node *head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) return head;

    struct node *p = head;
    struct node *q = NULL;
    struct node *r = NULL;

    while (p != NULL) {
        r = q;
        q = p;
        p = p->next;
        q->next = r;
    }

    head = q;
    return head;
}



struct node * insert_at_beginning(struct node *head, int x) {
    struct node *p = (struct node *)malloc(sizeof(struct node));
    p->data = x;
    p->next = head;
    head = p;
    return head;
}


struct node * insert_at_end(struct node *head, int x) {
    struct node *p = (struct node *)malloc(sizeof(struct node));
    p->data = x;
    p->next = NULL;

    if (head == NULL) {
        head = p;
        return head;
    }

    struct node *q = head;
    while (q->next != NULL)
        q = q->next;

    q->next = p;
    return head;
}


struct node * insert_before_y(struct node *head, int y, int x) {
    if (head == NULL) return NULL;

```

```c
    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = x;

    struct node *p = head, *q = NULL;

    while (p->next != NULL && p->data != y) {
        q = p;
        p = p->next;
    }

    if (p->data != y) {
        free(new_node);
        return head;
    }

    q->next = new_node;
    new_node->next = p;
    return head;
}


struct node * delete_from_end(struct node *head) {
    if (head == NULL) return NULL;
    struct node *p = head, *q = NULL;

    while (p->next != NULL) {
        q = p;
        p = p->next;
    }

    q->next = NULL;
    free(p);
    return head;
}



struct node * delete_x(struct node *head, int x) {
    if (head == NULL) return NULL;
    struct node *p = head, *q = NULL;

    while (p->next != NULL && p->data != x) {
        q = p;
        p = p->next;
    }

    if (p->data != x) return head;

    q->next = p->next;
    free(p);
    return head;
}


```

```c

struct node * find_middle(struct node *head) {
    if (head == NULL) return NULL;
    struct node *p = head, *q = head;

    while (q != NULL && q->next != NULL) {
        q = q->next->next;
        p = p->next;
    }

    return p;
}

int main() {
    struct node *head = NULL;
    head = insert_at_end(head, 10);
    head = insert_at_end(head, 20);
    head = insert_at_end(head, 30);

    struct node *p = find_middle(head);
}
```